Project1 RLOC

Mengqi Jia mj3664

November 13, 2025

1 Part 1

1.1 Discretize the system dynamics using Euler's method

The original quadrotor model is as follows:

$$\begin{aligned}
\dot{p}_x &= v_x \\
m\dot{v}_x &= -\left(u_1 + u_2\right)\sin\theta \\
\dot{p}_y &= v_y \\
m\dot{v}_y &= \left(u_1 + u_2\right)\cos\theta - mg \\
\dot{\theta} &= w \\
I\dot{w} &= r\left(u_1 - u_2\right)
\end{aligned}$$

The model can be discretized as follows:

$$\begin{aligned}
p_{x,n+1} &= p_{x,n} + v_{x,n}\Delta t \\
v_{x,n+1} &= v_{x,n} - \frac{\Delta t}{m}\left(u_{1,n} + u_{2,n}\right)\sin\theta_n \\
p_{y,n+1} &= p_{y,n} + v_{y,n}\Delta t \\
v_{y,n+1} &= v_{y,n} + \frac{\Delta t}{m}\left(u_{1,n} + u_{2,n}\right)\cos\theta_n - g\Delta t \\
\theta_{n+1} &= \theta_n + \omega_n\Delta t \\
\omega_{n+1} &= \omega_n + \frac{\Delta t}{I} r(u_{1,n} - u_{2,n})
\end{aligned}$$

According to the requirement of Project, we can define as follows the target trajectory so as to make the orientation θ perform at least 360-degree change:

$$\begin{aligned}
p_{x,t,tgt} &= 16\sin^3(t) \\
v_{x,t,tgt} &= \text{np.gradient}(p_{x,t,tgt}, \Delta t) \\
p_{y,t,tgt} &= 13\cos(t) - 5\cos(2t) - 2\cos(3t) - \cos(4t) \\
v_{y,t,tgt} &= \text{np.gradient}(p_{y,t,tgt}, \Delta t) \\
a_{x,t,tgt} &= \text{np.gradient}(v_{x,t,tgt}, \Delta t) \\
a_{y,t,tgt} &= \text{np.gradient}(v_{y,t,tgt}, \Delta t) \\
\theta_{t,tgt} &= -\text{np.unwrap}(\text{np.arctan2}(\text{a}_{\text{x,t,tgt}}, \text{a}_{\text{y,t,tgt}} + \text{g})) \\
\omega_{t,tgt} &= \text{np.gradient}(\theta_{t,tgt}, \Delta t)
\end{aligned}$$

Based on the above target trajectory, we can define the following cost function:

$$
\begin{aligned}
J(x,u) &= \frac{1}{2}\sum_{n=0}^{N}(x_n - x_{t,tgt})^T Q(x_n - x_{t,tgt}) + \frac{1}{2}\sum_{n=0}^{N-1} u_n^T R u_n \\
\text{subject to :} & 0 \le u_{1,n} \le 10, 0 \le u_{2,n} \le 10, n = 0, \ldots, N-1 \\
& p_{x,n+1} = p_{x,n} + v_{x,n}\Delta t \\
& v_{x,n+1} = v_{x,n} - \frac{\Delta t}{m}(u_{1,n} + u_{2,n})\sin\theta_n \\
& p_{y,n+1} = p_{y,n} + v_{y,n}\Delta t \\
& v_{y,n+1} = v_{y,n} + \frac{\Delta t}{m}(u_{1,n} + u_{2,n})\cos\theta_n - g\Delta t \\
& \theta_{n+1} = \theta_n + \omega_n \Delta t \\
& \omega_{n+1} = \omega_n + \frac{\Delta t}{I} r(u_{1,n} - u_{2,n}), n = 0, \ldots, N-1
\end{aligned}
$$

where

$$
\begin{aligned}
& Q = \mathrm{diag}(q_1, q_2, q_3, q_4, q_5, q_6), R = \mathrm{diag}(r_1, r_2) \\
& x_n = (p_{x,n}, v_{x,n}, p_{y,n}, v_{y,n}, \theta_n, \omega_n), u_n = (u_{1,n}, u_{2,n}) \\
& x_{t,tgt} = (p_{x,t,tgt}, v_{x,t,tgt}, p_{y,t,tgt}, v_{y,t,tgt}, \theta_{t,tgt}, \omega_{t,tgt}) \\
& t = n\Delta t, \Delta t = \frac{duration}{N}, N = time\ steps
\end{aligned}
$$

1.2 To implement SQP to solve the problem

We need to take the following steps to implement SQP:

- 1. The Lagrangian function and Hessian matrix are as follows:

$$
\begin{aligned}
& \mathcal{L}(X, \lambda, \mu) = J(X) + \lambda^T g(X) + \mu^T h(X) \\
& H = \nabla^2_{XX} J(X) \approx \nabla^2_{XX} \mathcal{L}(X) \\
& G = \nabla_X g(X) \\
& G_{ineq} = \nabla_X h(X)
\end{aligned}
$$

  where $X = [x_0, u_0, \ldots, x_{N-1}, u_{N-1}, x_N]$, and the second order derivative of the constraints are ignored.

- 2. We can obtain the following SQP problem:

$$
\begin{aligned}
& \min_{p_X} p_X^T \nabla^2_{XX} \mathcal{L}(X) p_X + p_X^T \nabla f(X) \\
\text{subjuec to } & \nabla_X g(X)^T p_X + g(X) = 0 \\
& \nabla_X h(X)^T p_X + h(X) \le 0
\end{aligned}
$$

- 3. When deciding if a step is acceptable, we follow this rule:
  (1) implement a function that computes the amount of nonlinear constraint violation, i.e., we compute the absolute value of all nonlinear constraints $|c(x)|$;
  (2) run the QP solver for 100 iterations, and implement the simplified filter line search:
  1. set $\rho = 0.5$ and $\alpha = 1$;
  2. initialize $f_{best} = \infty$ and $c_{best} = \infty$, and if $f(x_{guess} + \alpha step) < f_{best}$ or $|c(x_{guess} + \alpha step)| < c_{best}$ then accept the step and update f_{best} and c_{best}; if the step was not accepted then set $\alpha \leftarrow \rho\alpha$ and repeat until it gets accepted;
  3. update the state as $x_{guess} \leftarrow x_{guess} + \alpha step$; terminate the algorithm when either reached the maximum number of iterations or the KKT optimality conditions are satisfied within a small margin, e.g. tol=10^{-4}.

The results are as follows:
(1) The plot of α, constraint violation, and cost per iteration:

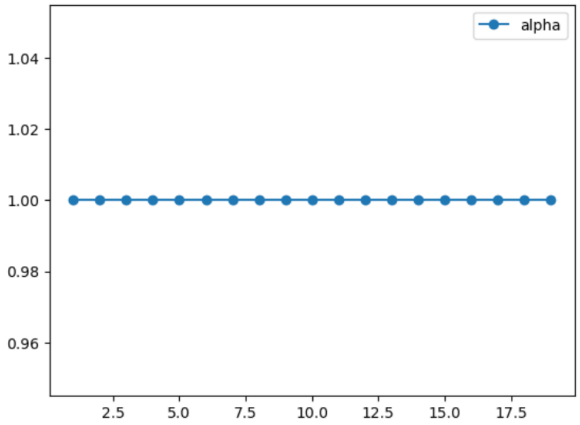


Figure 1: The change of α per iteration

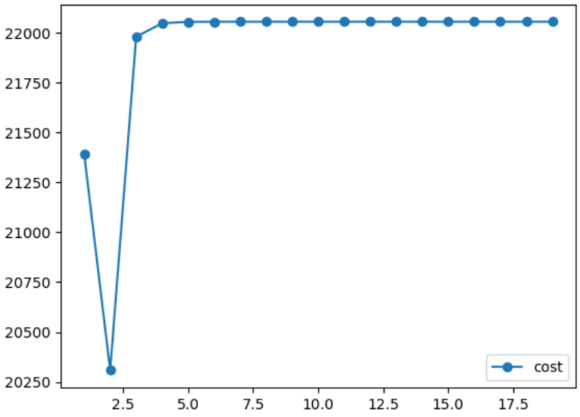


Figure 2: The change of cost per iteration

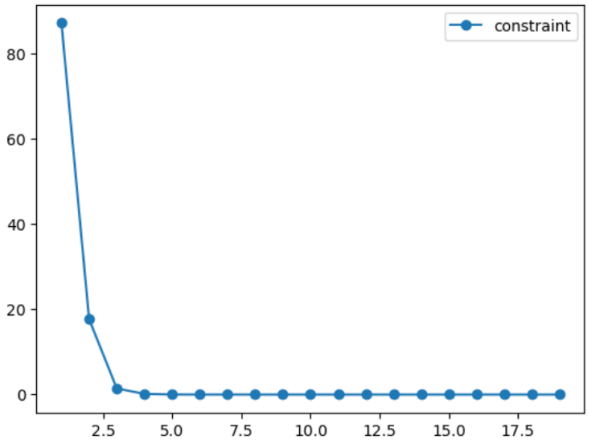


Figure 3: The change of constraint violation per iteration

(2) The plot of states and controls of the robot:

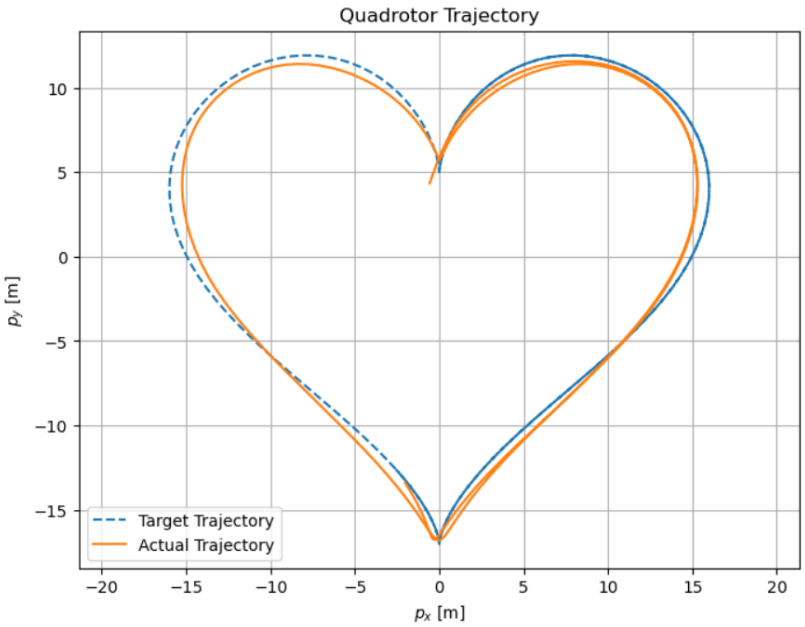


Figure 4: The trajectory of the robot

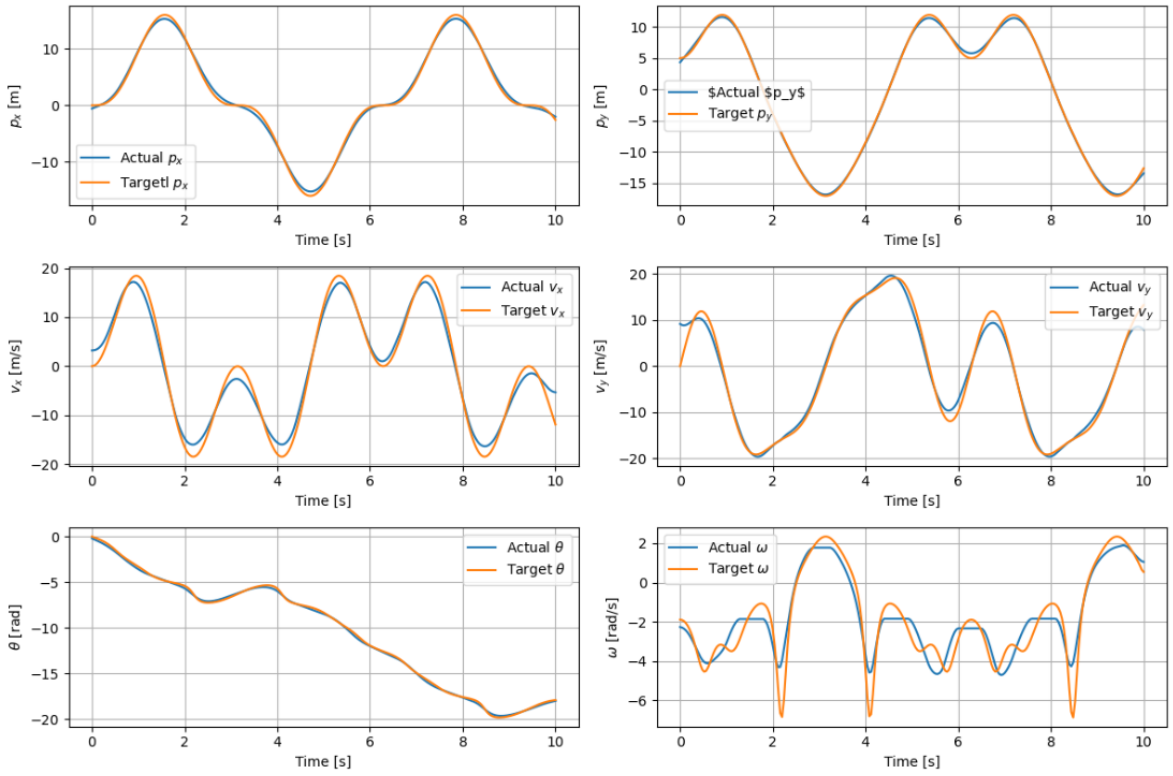


Figure 5: The states of the robot

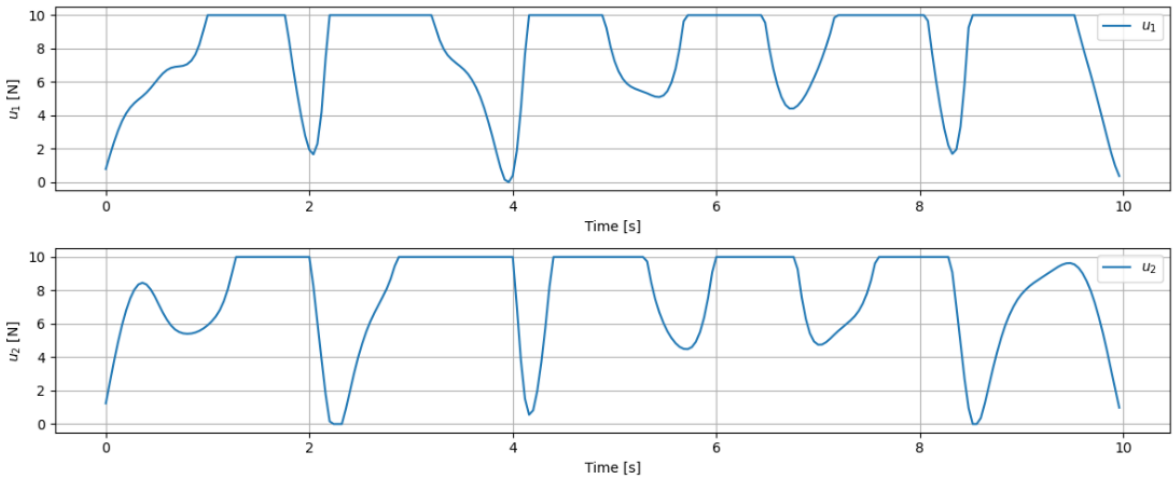


Figure 6: The controls of the robot

2 Part 2

In this section, we use trajectory optimization method from Part I to design a MPC controller and test it using the simulator. The discretization of model and cost function are the same as Part I. Here we just briefly introduce some key steps for MPC design.

2.1 MPC design

- 1. Determine the horizon window for optimization as $N_p = \min(50, N - t_{current})$, where N is the total time step and $t_{current}$ is the current time step;
- 2. For faster convergence, we initialize the states to be optimized with the current state and target trajectory in N_p step;
- 3. use QP solver to solve the optimization problem in N_p step and use the first control law and simulate function to predict the next step, and use it as the starting state for the next optimization process;
- use the same line search as Part I to decide if an optimized step is accepted, and also use the same exit rule as Part I.

The results are as follows(with disturbance):

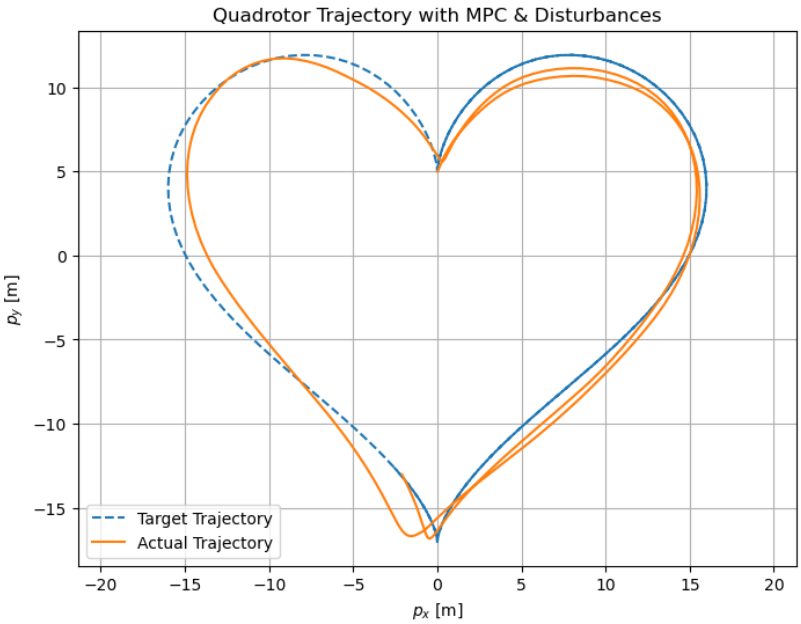


Figure 7: The states of the robot

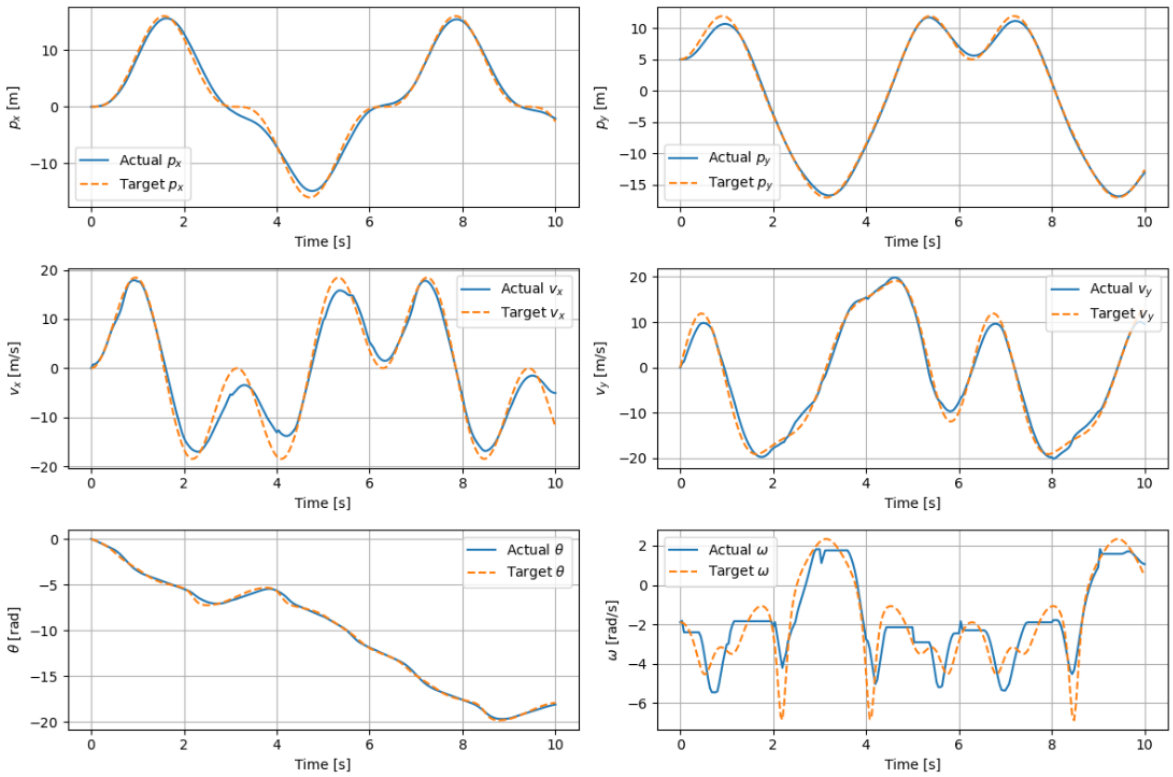


Figure 8: The states of the robot

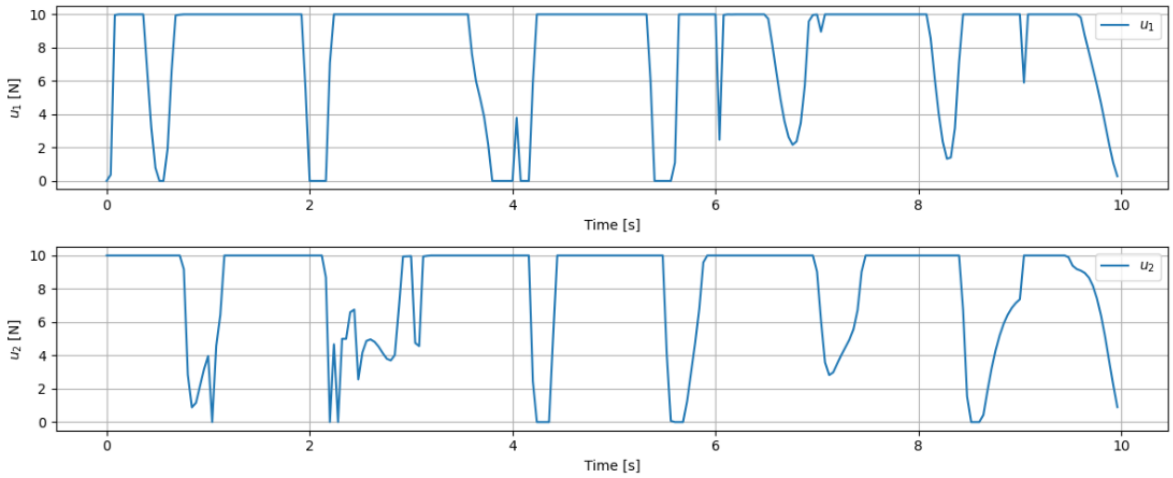


Figure 9: The controls of the robot